

FACULTAD DE CIENCIAS EXACTAS UNICEN

Introducción a Python

Clase Práctica Nro. 1

¿Qué vamos a ver hoy?

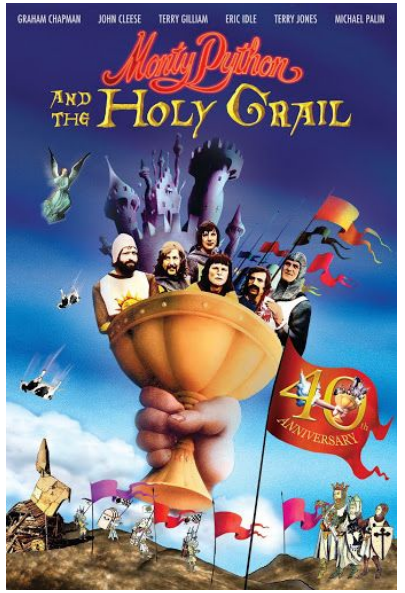
Curso acelerado (aceleradísimo) de Python para gente que ya programa (y bien!)

- Motivos para usar Python en esta materia en lugar de otros lenguajes.
- Diferentes formas de utilizar Python (usando scripting o Jupyter Notebooks, localmente o Google Colab).
- Sintaxis principal de Python y sus características distintivas.
- Proceso de instalación de Python en sus compus.



¿Por qué Python?

- Un poquito de historia
  - Creado por Guido van Rossum (Países Bajos) y lanzado por primera vez en 1991.
  - **Objetivo**: hacer un lenguaje fácil de leer
  - El nombre viene del grupo de comedia “Monty Python”
  - 3 versiones, la más nueva (y la que vamos a usar nosotros!) es Python 3
- Python es **popular** y tiene una gran comunidad de desarrolladores y recursos disponibles
  (incluyendo foros y obviamente ChatGPT + Copilot + todas las AI para softdev)
- Es el **lenguaje más utilizado para ciencia de datos** por su simplicidad y por las librerías disponibles para trabajar



Características principales de Python

- Lenguaje de **propósito general**, de **alto nivel**
- Ámbitos determinados **por indentaciones**
- **Tipos dinámicos** de datos
- Garbage collector propio
- **Multiparadigma** (procedural, orientado a objetos, funcional)



Es muy fácil programar horrible en Python

POR FAVOR, PROGRAMEMOS BIEN

(esto incluye leer la consola cuando hay un error, incluso antes de hablar con ChatGPT ;-)

¿Cómo podemos trabajar con Python?



Local

- El código corre directamente en sus máquinas
- No necesitan internet para ejecutarlo
- Pueden usar el IDE que quieran (VSCode, Pycharm, ...)
- Los pasos de instalación dependerán de si usan Windows, Linux o macOS * (revisen bien porque a veces ya está instalado!)
- Indispensable usar un gestor de ambientes virtuales para evitar problemas

Web

(Ej. Google Colab)

- El código corre en un servidor remoto de Google
- Se trabaja a través de Google Colab (Jupyter notebook)
- Necesitás internet para trabajar
- No hace falta instalación.
- Cada Colab es un ambiente virtual
- Nuestros ejemplos están en Colab para más facilidad de uso

* Les recomendamos que usen Linux (cualquier distro) o macOS, pero pueden instalar Python también en Windows

¿Cómo podemos trabajar con Python?



Local

- El código corre directamente en sus máquinas
- No necesitan internet para ejecutarlo
- Pueden usar el IDE que quieran (VSCode, Pycharm, ...)
- Los pasos de instalación dependerán de si usan Windows, Linux o macOS * (revisen bien porque a veces ya está instalado!)
- Indispensable usar un gestor de ambientes virtuales para evitar problemas

Web
(Ej. Google Colab)

- El código corre en un servidor remoto de Google
- Se trabaja a través de Google Colab (Jupyter notebook)
- Necesitás internet para trabajar
- No hace falta instalación.
- Cada Colab es un ambiente virtual
- Nuestros ejemplos están en Colab para más facilidad de uso

* Les recomendamos que usen Linux (cualquier distro) o macOS, pero pueden instalar Python también en Windows

Introducción a Python

Google Colab



- La herramienta que vamos a usar para ejemplos y explicaciones en clase.
- Jupyter Notebooks que corren en la nube de Google (no en tu compu!).
  - Totalmente integrado a Google Drive.
    - Accede a tu unidad de Google Drive como si fuera el disco de tu compu.
    - Se guardan (acordate de Ctrl+S de vez en cuando) en tu Google Drive, en la carpeta Colab Notebooks.
- Acordate que en tu cuenta @alumnos.exa.unicen.edu.ar (casi) no tenés limitaciones de espacio en Google Drive → Usá esa!

NO ENTREGUEN EL TRABAJO ESPECIAL EN COLAB!!!

Lo tienen que hacer usando Jupyter Notebooks, y entregarlo como repositorio en Github



[ciencia-datos-unicen-practica-1-python.ipynb]

Y ahora... tutorial rápido de Python

En el Google Colab de la clase de hoy, veremos:

1. **Un primer Hola Mundo**: para empezar a manejar la sintaxis
2. **Manejo de variables**: para entender los tipos principales, y cómo usar variables dinámicas
3. **Funciones:** cuál es la sintaxis principal para escribir funciones
4. **Estructuras de control:** el if, el for y el while.
5. **Estructuras de datos principales:** la lista, la tupla y el diccionario.
6. **Manejo de excepciones:** cómo hacer un try / except en Python.
7. **Clases y objetos:** para que sigan programando orientado a objetos!
8. **Diferentes formas de importar librerías.**



[ciencia-datos-unicen-practica-1-python.ipynb]

Instalación de Python



Local

- El código corre directamente en sus máquinas
- No necesitan internet para ejecutarlo
- Pueden usar el IDE que quieran (VSCode, Pycharm)
- Los pasos de instalación dependerán de si usan Windows, Linux o macOS * (revisen bien porque a veces ya está instalado!)
- Indispensable usar un gestor de ambientes virtuales para evitar problemas

Web

(Ej. Google Colab)

- El código corre en un servidor remoto de Google
- Se trabaja a través de Google Colab (Jupyter notebook)
- Necesitás internet para trabajar
- No hace falta instalación.
- Cada Colab es un ambiente virtual
- Nuestros ejemplos están en Colab para más facilidad de uso

* Les recomendamos que usen Linux (cualquier distro) o macOS, pero pueden instalar Python también en Windows

¿Qué necesito para usar Python local?

Instalar Python en tu computadora	**Crear tu ambiente virtual (virtual environment)** **[Opcional pero recomendado]**	**Instalar un IDE para escribir el código**	**Configurar la terminal**
Instrucciones para Windows, Linux o macOS	Usando PIP o Conda (o pyenv)	VSCode, Pycharm, “Jupyter”, ~~Spyder~~, ~~VIM~~	Abrir terminal, activar el venv y a codear y correr!

Gestión de ambientes virtuales

- Python está **basado en librerías** que se interrelacionan entre sí con diferentes versiones
- Es **fundamental aislar cada proyecto** para evitar que las (versiones de las) librerías de un proyecto no rompan las librerías de los demás

Ambientes virtuales

(virtual environments, venvs)

- Una especie de “máquina virtual de paquetes”, donde **coexisten diferentes librerías asociadas a un proyecto**.
- Los ambientes están aislados entre sí, lo que permite que puedas trabajar con diferentes versiones de una misma librería en más de un proyecto, sin problema.
- Es una **práctica estándar** (y muy buena) crear un ambiente virtual al comenzar un proyecto



Gestores de ambientes virtuales (venv)

PIP (Python Package Installer)	Conda
Gestor de paquetes oficial para Python	Simplificación de PIP, construido sobre PIP
No requiere instalación (instalado por defecto en Python)	Necesitás instalar Anaconda (Windows - Linux - macOS)
Instala paquetes del Python Package Index (PyPI) (no precisás decirle de dónde traerlos)	Instala paquetes desde la distribución de Anaconda y de otros canales (le tenés que decir de dónde)
Se conecta con virtualenv para construir ambientes virtuales	Tiene esta característica ya provista internamente (built-in)
Todas las librerías cuelgan de la misma versión de Python	Cada ambiente puede tener su propia versión de Python
Instala los paquetes desde código fuente, los compila al instalarlos (proceso más lento, puede fallar, sobre todo en Windows)	Los paquetes que instala son binarios compilados ya para la arquitectura donde van a correr (la instalación suele ser más rápida, resuelve conflictos en el camino)
Te dejamos acá un tutorial	Te dejamos acá un tutorial

Más info para lxs manija:
https://medium.com/analytics-vidhya/understand-conda-and-pip-9e5c67da47cc - https://www.anaconda.com/blog/understanding-conda-and-pip

Crear/borrar venvs con PIP



Para **crear un ambiente virtual**, podés el comando venv de virtualenv:

```
python -m venv <nombre del ambiente>
```

- OJO: Se te crea en la carpeta donde estés - Asegurate de ponerlo en el .gitignore de tu proyecto para evitar agregarlo al repo.
- Si no te corre, chequeá que tengas virtualenv instalado en tu versión de python, haciendo `pip install virtualenv`

Para **borrar un ambiente virtual** (si lo creaste mal, por ejemplo), no hay un comando predefinido, tenés que borrar la carpeta que se creó.

Ejemplo en macOS / UNIX (desde la terminal, parado en la carpeta donde está tu venv)

```
sudo rm -rf <nombre del ambiente>
```

Activar/desactivar venvs con PIP



Para **activar** **un venv**, en una terminal, parados en la carpeta donde está el ambiente:

En Windows:

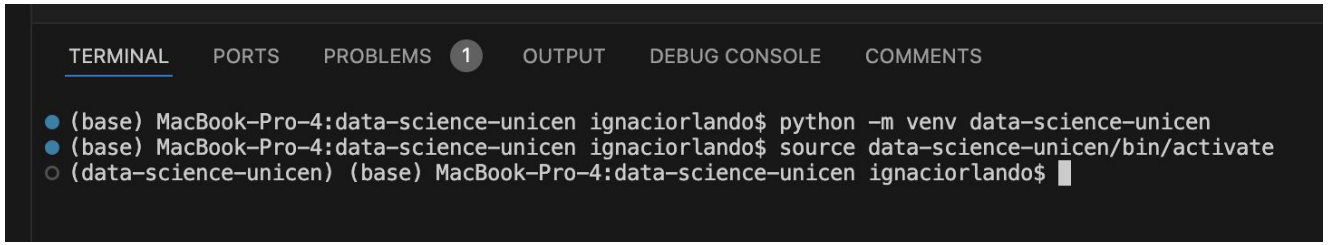
```
<nombre del ambiente>\Scripts\activate
```

En macOS / UNIX:

```
source <nombre del ambiente>/bin/activate
```

Ojo con olvidarse del venv y terminar instalando paquetes en el root!

Cuando está activado, en la terminal tenés que ver: `(nombre del ambiente)` *antes del cursor*



Para **desactivar** **un venv**, parados en la carpeta donde está el ambiente:

En cualquier SO:

```
deactivate
```

(Des)instalar paquetes con PIP



Para **instalar un paquete** **en un venv**, en una terminal, parados en la carpeta donde está el ambiente, **con el ambiente activado**:

```
pip install <nombre del paquete>
```

Para **desinstalar un paquete** **en un venv**, en una terminal, parados en la carpeta donde está el ambiente, **con el ambiente activado**:

```
pip -m uninstall <nombre del paquete>
```

Para **actualizar la versión de un paquete** que ya tenés en un venv, en la misma terminal y con el ambiente activado, corrés:

```
pip install <nombre del paquete> --upgrade
```

Compartir ambientes con PIP



Con PIP podemos **empaquetar un venv** **para poder compartirlo**. Esto facilita banda que otros usuarixs puedan levantar tu código, correrlo y hacer contribuciones.

```
pip freeze > requirements.txt
```

- Se genera un archivo requirements.txt en el directorio en el que estés paradx en la terminal (que podés/debés gitear!)
- Dentro del archivo figuran todas las librerías que tenés instaladas y su versión.

Para **instalar una versión empaquetada** **de un venv**, tenés que correr:

```
pip install -r requirements.txt
```

Crear/borrar venvs con Conda



Para **crear un ambiente virtual**, en una terminal ejecutás:

```
conda create --name <nombre del ambiente>
```

- Podés usar `-n` en vez de `--name` (los dos guiones se usan para indicar el nombre completo del parámetro, con el guión solo se logra lo mismo pero sin tener que escribir el nombre entero)
- Puede que conda te pregunte si querés seguir o no, con un `proceed ([y]/n)?`. Si pasa eso, tipeá `y`.
- Para crear un ambiente virtual configurado para una versión específica de Python, se lo indicás haciendo:
  `conda create -n <nombre del ambiente> python=<version>`

Para **listar los ambientes virtuales** que creaste, ejecutás:

```
conda info --envs
```

Para **borrar un ambiente virtual** (si lo creaste mal, por ejemplo), ejecutás:

```
conda remove --name <nombre del ambiente> --all
```

- Para chequear si está bien borrado, listá los ambientes y fijate si el nombre sigue apareciendo.

Activar/desactivar venvs con Conda

Para **activar un venv**, en una terminal, ejecutás:

```
conda activate <nombre del ambiente>
```

Cuando está activado, en la terminal tenés que ver `(nombre del ambiente)` *antes del cursor*

Para **desactivar un venv**, en una terminal, ejecutás:

```
conda deactivate
```

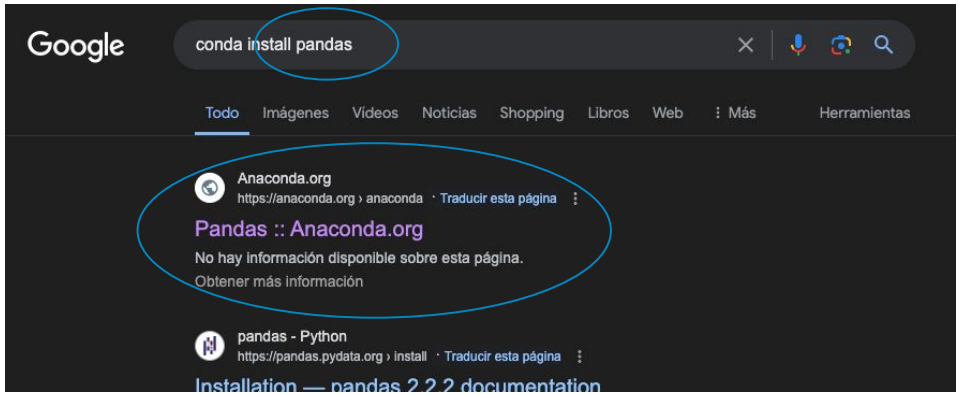
Cuando está desactivado, en la terminal tenés que ver que ya no está el nombre del ambiente (puede leerse `(base)`*, pero no tu ambiente)*

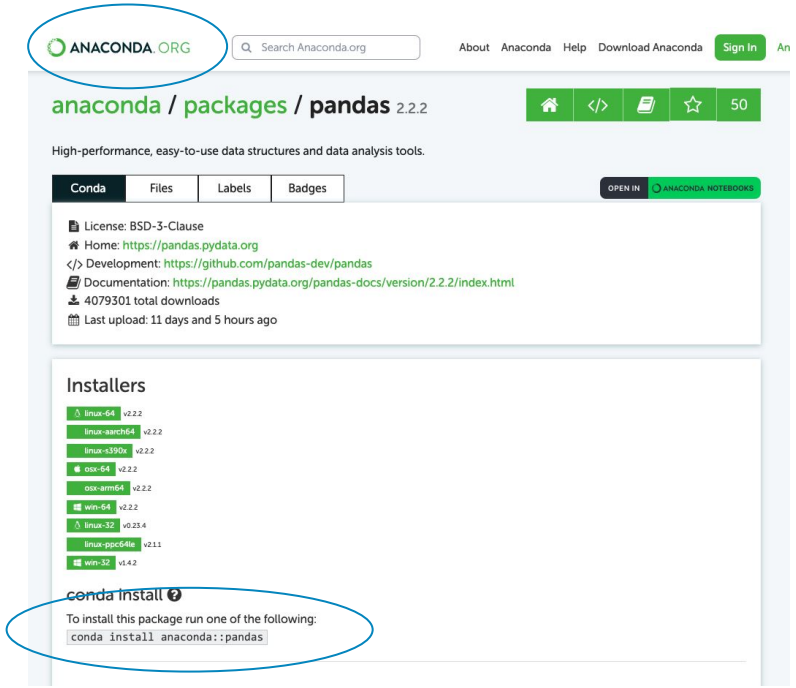
Ojo con desactivar el ambiente, olvidarse y terminar instalando paquetes en el root!

(Des)instalar paquetes con Conda

Para **instalar un paquete** **en un venv**:

1. Buscá en Google “<nombre del paquete> install conda”





1. En una terminal, parados en la carpeta donde está el ambiente, **con el ambiente activado**:

   `conda install` <y lo que dice la web de conda que hagas!>

(Des)instalar paquetes con Conda

Para **desinstalar un paquete** **en un venv**, en una terminal, parados en la carpeta donde está el ambiente, **con el ambiente activado**, corremos:

```
conda remove <nombre del paquete>
```

Con Conda también podemos **empaquetar un venv** **para poder compartirlo**, pero en vez de hacerlo en un requirements.txt se hace en un environment.yml.

```
conda env export > environment.yml
```

- Este archivo tiene nombres y versiones de las librerías **para tu sistema operativo**, con lo cual si lo vas a compartir con usuarios que usar otro SO, no va a andar :-/

Para **crear un venv a partir de su versión empaquetada**, tenés que correr:

```
conda env create -f environment.yml
```

Probemos si todo anda bien!

1. Abrir una terminal.
2. Activar el venv que hayan creado.
3. Correr el comando `python`. Automáticamente, la consola es un intérprete de Python. Si no se abrió, hubo un problema con la instalación de Python.
4. Dentro de esa consola:
   a. Pueden chequear que Python esté andando escribiendo el clásico “Hola mundo!”

      ```
      print(“Hola mundo!”)
      ```

   b. Pueden probar si se instaló bien una librería haciendo:

      ```
      import <nombre de la librería>
      <nombre de la librería>.__version__
      ```

Los dos sabores de Python para DS

Python scripting

- La programación típica de siempre
- Escribimos instrucciones en archivos .py agrupados en carpetas
- Ejecutamos los mains desde la terminal



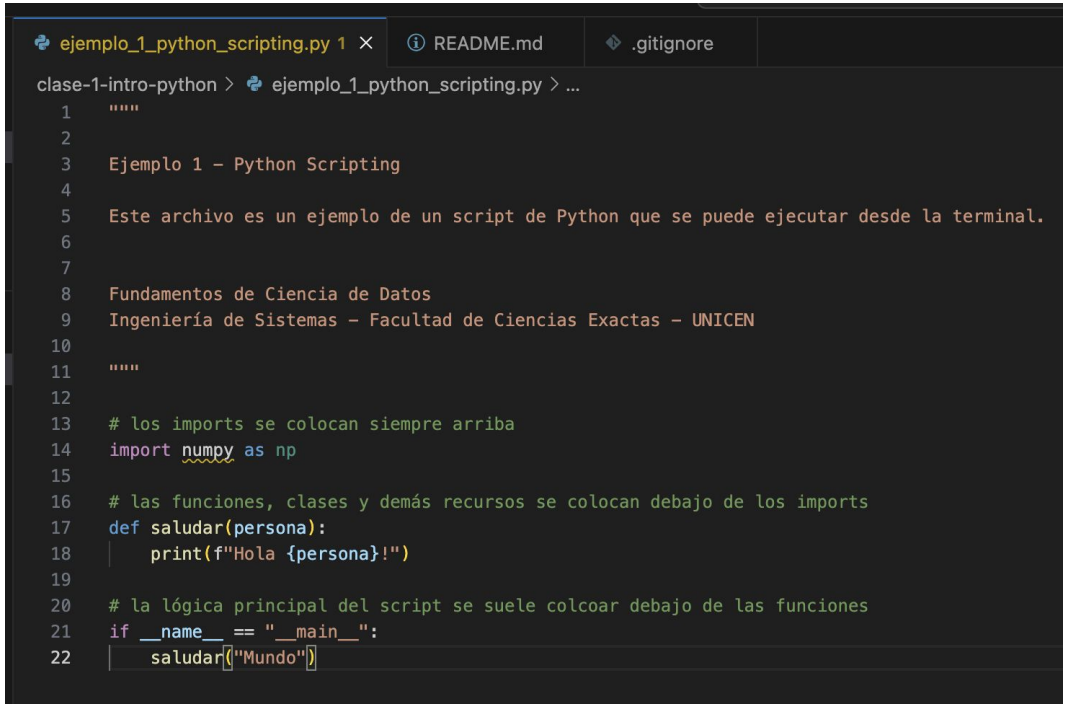
Jupyter notebooks

- Ju-lia, Py-thon, (e)R: el núcleo de los 3 es la ciencia de datos
- Un ambiente interactivo que se ejecuta en el navegador
- Nos permite escribir y ejecutar código a la vez, en celdas.
- Permite combinar código, ecuaciones, visualizaciones, texto narrativo, etc.
- **Vamos a usar esta herramienta para los ejemplos que les mostremos durante las clases, en su versión de Google (Colabs)**

Python scripting

- Se crea un archivo .py con el main y el código a ejecutar.
  - Lo ideal es que usen un IDE para esto (VSCode)
- Ideal que lo usen para automatizar tareas que se ejecutan en lotes y que no necesitan interacción del usuario.
- Un ejemplo de script
- Para correr el script:

  `python <nombre archivo>`

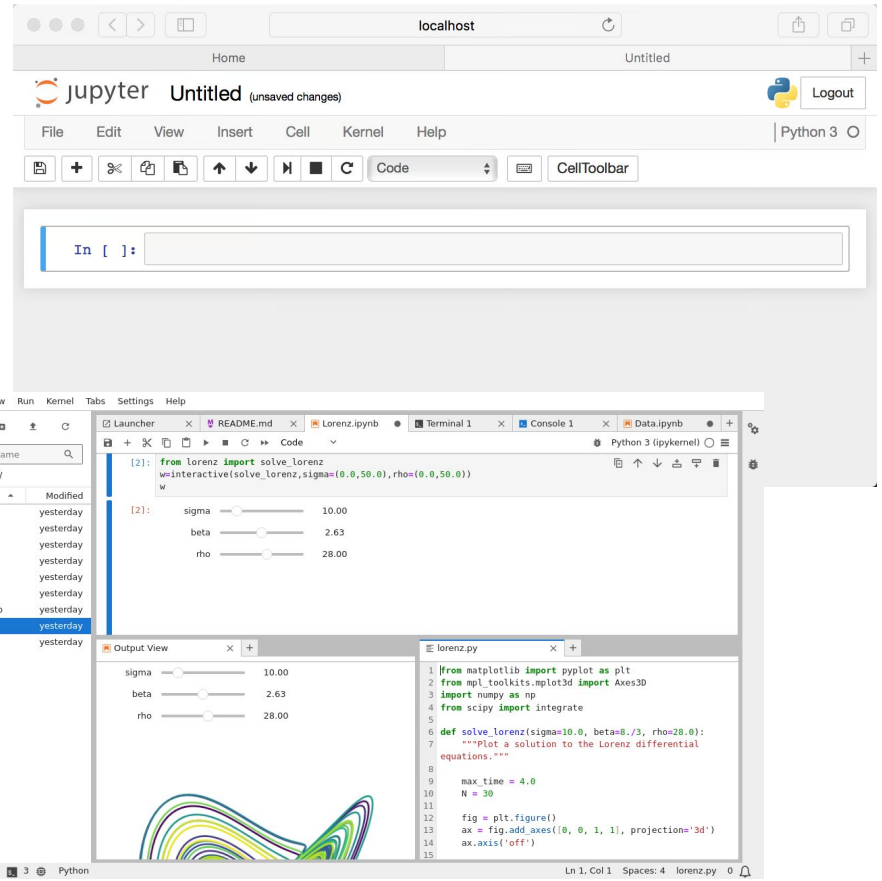


Jupyter notebooks

- Requisitos previos: instalar jupyter!
  - Recuerden hacerlo dentro de un ambiente virtual ya creado.
  - Alternativa (recomendada): jupyterlab
    - Mejor UI, algunas funciones más.
    - `pip install jupyterlab`
- Para iniciar una notebook:
  - En jupyter:

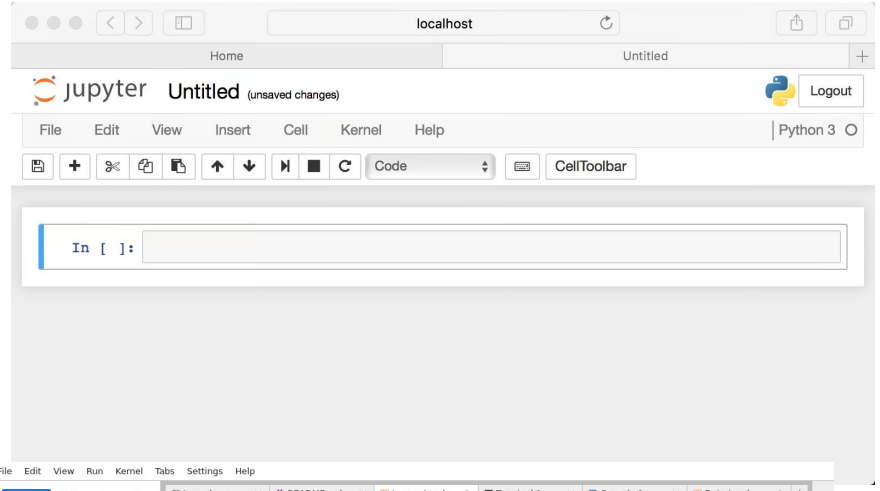
    `jupyter notebook`
  - Con jupyterlab:

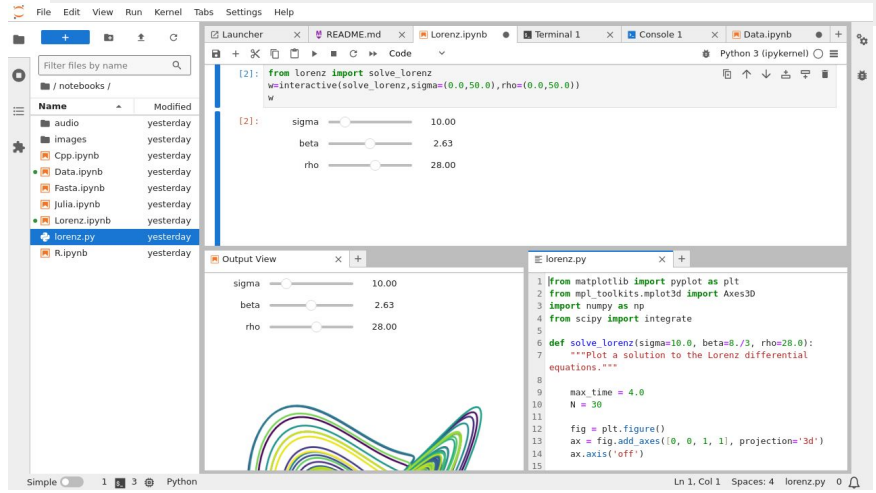
    `jupyter-lab`
- Vamos con un ejemplo



Jupyter notebooks - Algunos consejos

- Nunca notebooks para código productivo (pésima práctica)
  - Poca trazabilidad
  - Demasiado dinámico
    - Cada vez que hagan una corrida diferente y cambie la salida, se va a tomar como un cambio en el código. → Ojo con Git
  - Ojo con la memoria:
    - Las variables declaradas siguen quedando disponibles, ante la duda reinicien el entorno de ejecución
- Usos recomendados:
  - Experimentos / pruebas primarias
  - Demos para stakeholders
  - **Entrega del trabajo especial**





Los dos sabores de Python para DS

Diferencias e/ scripting y notebooks

Scripting	Notebooks
Tenemos que usar un IDE (VSCode, PyCharm) o un editor de texto	Ambiente de desarrollo interactivo dentro del navegador web
Se ejecuta un script completo	El código se agrupa en celdas, mezclando texto y código
Para debuggear necesitamos usar las herramientas del IDE	Podemos debuggear aprovechando las celdas
Se documenta solamente a través de comentarios en linea o docstrings	Además de lo anterior, podemos agregar celdas con texto en Markdown y dejar imágenes o las salidas visuales
Los gráficos y demás salidas se abren en ventanas aparte.	Los gráficos se ven secuencialmente en la misma notebook.
Es más fácil de usar con Git	Es un bardo con Git (cada vez que hay una salida nueva, cambia el archivo)
Necesito tener una función main que se ejecute	Necesito abrir la notebook en Jupyter (o Colab) y listo
Te dejamos acá un tutorial completo	**Te dejamos acá un tutorial completo (recomendado!)**

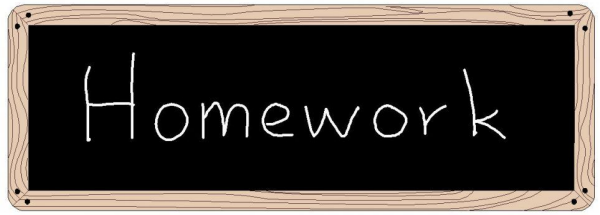
¿Qué vimos hoy?

1. Motivos para usar Python en esta materia en lugar de otros lenguajes.
2. Diferentes formas de utilizar Python (usando scripting o Jupyter Notebooks, localmente o Google Colab).
3. Sintaxis principal de Python y sus características distintivas.
4. Proceso de instalación de Python en sus compus.

Material adicional:

- Instalar y configurar Python en Visual Studio Code (ojo con la versión de Python)
- Python desde cero: 8 horas de un curso acelerado de Python.
- Tutorial de instalación de librerías para Python usando PIP.
- Capítulo 2 del libro de Joel Grus.

Instalación de Python



Para la clase que viene:

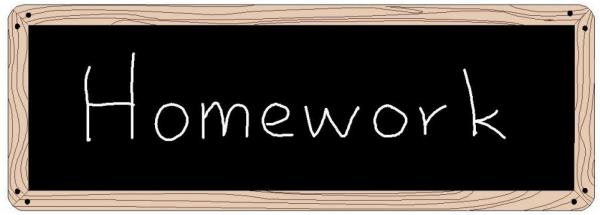
1. Instalar Python en la computadora sobre la que van a hacer el TPE
2. Verificar la instalación.
3. Generar el entorno virtual (virtual environment) para su proyecto (puede ser usando PIP o usando Conda, lo que prefieran)
4. Crear el espacio para el proyecto en Github (público), con un `.gitignore` para Python (así no suben entornos virtuales ni archivos auxiliares)
5. Clonar el repositorio en sus computadoras.
6. Instalar Jupyter en el venv



Si tienen dudas sobre cómo usar github, usen: https://docs.github.com/es/get-started/start-your-journey/hello-world

Introducción a Python

Trabajo Práctico Nº 1



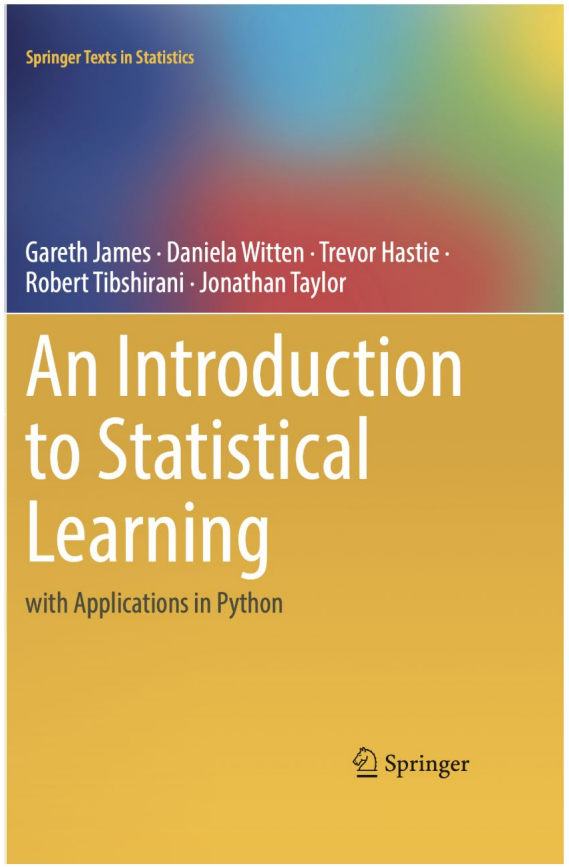
Resolver los siguientes ejercicios de programación básica utilizando Python. Les recomendamos que trabajen en notebooks de Júpiter propias en su propia computadora, o bien con scripts, para familiarizarse con el lenguaje corriendo local. De todas formas, si quieren usar Google Colab, pueden hacerlo.

Recuerden que este práctico no se entrega, tiene como propósito solamente que se familiaricen con el lenguaje.

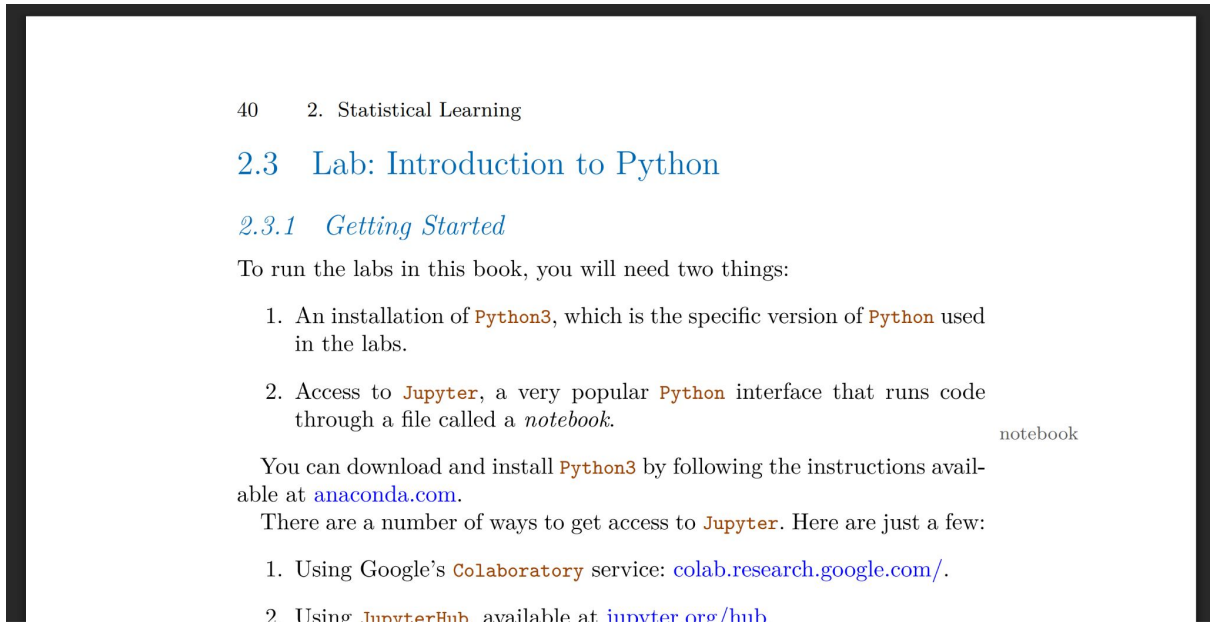
1. Escribir un programa que pida al usuario su nombre y lo salude. Pista: van a necesitar la función input().
2. Crear una función que devuelva la suma de dos números.
3. Escribir un programa que calcule el área de un círculo dado su radio.
4. Crear una lista de 5 frutas y escribir un bucle que las imprima una por una.
5. Escribir una función que determine si un número es par o impar.
6. Crear un diccionario con nombres de estudiantes y sus notas. Imprimir el promedio de esas notas.
7. Escribir un programa que genere una lista de los primeros 20 números de la secuencia de Fibonacci.
8. Crear una función que reciba una cadena y devuelva True si es un palíndromo. Recordar que un palíndromo es aquella palabra que, espejada, se sigue leyendo igual (por ejemplo, neuquen).
9. Crear una clase Rectángulo con atributos de ancho y alto, y métodos para calcular su área y perímetro.
10. Crear una clase Cuadrado que herede de Rectángulo y devuelva las mismas propiedades.

Introducción a Python

Para los más manija



https://www.statlearning.com/



40 2. Statistical Learning

2.3 Lab: Introduction to Python

2.3.1 Getting Started

To run the labs in this book, you will need two things:

1. An installation of `Python3`, which is the specific version of `Python` used in the labs.
2. Access to `Jupyter`, a very popular `Python` interface that runs code through a file called a *notebook*.

notebook

You can download and install `Python3` by following the instructions available at anaconda.com.

There are a number of ways to get access to `Jupyter`. Here are just a few:

1. Using Google's `Colaboratory` service: colab.research.google.com/.
2. Using `JupyterHub` available at jupyter.org/hub

Capítulo 2 tienen un montón más de contenido sobre uso de Python y ejemplos. **En la sección 2.4, tienen una lista de ejercicios (a partir del 8)**